Seminar for Master Students in Software Engineering
(Software Engineering and Programming)

Master Thesis Proposal Draft

Behavioral Analysis of Client-Side Manipulation Techniques in Games

Student:
Alexander Spiesberger, 12334182

April 30, 2026

Advisor:
Prof. Martin Ertl

1 Problem Statement and Motivation

Modern interactive systems such as online games rely on client-side execution for responsiveness, but this exposes program state and execution logic to users who control the client machine, enabling manipulation techniques such as memory modification, code injection, and execution-flow manipulation. Existing defenses often rely on signatures, heuristics, or integrity checks that focus on known implementations and can be bypassed as techniques evolve. At the same time, many manipulation techniques, despite differing in implementation, rely on a limited set of underlying operating-system primitives such as process memory access, module loading, and thread creation. This raises the question whether these techniques exhibit stable patterns at the system level that are independent of specific implementations. While prior work has studied process injection, runtime anomaly detection, and anti-cheat defenses, it remains insufficiently understood which operating-system-level effects are consistently produced by client-side manipulation techniques, which remain observable under evasion strategies, and whether such effects can distinguish manipulated from benign execution.

This thesis addresses the following research question:

> *To what extent can common client-side manipulation techniques be characterized and distinguished through their observable operating-system-level behavior?*

Answering this question is relevant because it can improve understanding of the fundamental properties shared across manipulation techniques and provide foundations for more robust behavior-based detection approaches that are less dependent on specific known attacks.

2 Aim of the Thesis

The goal of this thesis is to empirically investigate whether client-side attacks exhibit stable and distinguishable patterns at the operating-system level. To achieve this, the thesis will first classify representative approaches based on the system primitives they rely on. It will then analyze the effects these approaches produce during execution and evaluate which of these effects remain observable across different implementations and under evasion. By identifying such stable indicators, the thesis aims to provide empirical foundations for behavior-based detection and address the limitations of implementation-specific defenses discussed in Section 1.

This goal is structured around the following research subquestions:

1. Which operating-system primitives and observable effects are shared across representative classes of attacks?
2. To what extent do different implementations of similar approaches exhibit consistent execution patterns?

3. How do evasion strategies influence the observability and stability of these patterns?

4. To what extent can observed effects distinguish manipulated from benign execution and differentiate between classes of attacks?

3 Methodology and Expected Results

This thesis combines analytical and experimental methods to study how representative attacks manifest at the operating-system level. It first analyzes publicly available technical sources and existing tools to identify common approaches and the system primitives they rely on, such as memory access, code injection, execution control, and evasion. Based on this analysis, the thesis selects representative approaches and implements them as prototypes for controlled experiments. It then observes their execution in a monitored Windows-based environment that captures system-level events, including memory and protection changes, thread activity, and process interactions. The thesis compares the resulting traces to identify recurring patterns, evaluate how consistently they appear across different implementations, and analyze how evasion strategies affect their visibility.

The research subquestions from Section 2 are addressed through this experimental analysis: (1) shared effects are studied by comparing different classes of attacks and identifying common patterns; (2) consistency is evaluated by implementing similar approaches in different ways and analyzing how often the same patterns appear; (3) robustness is investigated by introducing representative evasion techniques and measuring which effects remain observable; and (4) distinguishability is assessed by comparing attack traces with benign executions and evaluating how clearly they differ. Based on this analysis, the work derives a classification of representative approaches in terms of their underlying system primitives and provides an empirical characterization of patterns that remain observable across implementations. The evaluation focuses on measurable properties such as the frequency with which patterns recur, the extent to which indicators remain visible under evasion, and the degree to which attack traces differ from benign execution. These aspects are assessed through comparative trace analysis using simple quantitative measures, including event frequencies, recurring patterns, and observable differences between traces.

4 State of the Art

Client-side manipulation techniques have primarily been studied in the context of malware analysis and, more recently, in game security. One important line of research focuses on behavioral detection at the system level. Forrest et al. [4] showed that normal program execution follows relatively stable system-call patterns and that deviations can indicate abnormal behavior. Building on this idea, Canzanese et al. [1] use system-call traces to characterize and distinguish malicious processes, demonstrating that system-level sig-

nals can capture relevant properties of program execution. Complementary to this, other work focuses on the attack mechanisms themselves. Di Pietro et al. [3] analyze Windows process injection by identifying shared low-level primitives across different approaches, providing a structured understanding of their underlying mechanisms. Nasereddin and Al-Qassas [5] study memory-level artifacts produced during such attacks, highlighting observable effects at a finer granularity. In the context of games, Collins et al. [2] examine client-side defenses and emphasize both the relevance of the problem and the limitations of current anti-cheat mechanisms.

While these works provide important building blocks, they remain largely focused on either observable signals (e.g., system calls or memory artifacts) or on the structure of individual attack techniques. As a result, these perspectives are not systematically connected. In particular, it remains unclear how different classes of client-side manipulation techniques translate into observable system-level effects during execution, how consistent these effects are across different implementations, and whether they can serve as reliable and generalizable indicators of manipulation. This means that the research question of this thesis is non-trivial. Although both relevant signals and underlying mechanisms are known, it is still unclear which system-level effects consistently occur across different techniques and implementations, remain observable under evasion, and allow a reliable distinction between manipulated and benign execution. This thesis addresses this gap by analyzing multiple representative manipulation techniques at the operating-system level and observing their runtime effects. It compares these effects across different implementations and evaluates how consistently they appear, how they are affected by evasion, and how clearly they differ from benign behavior. The goal is to identify observable indicators that are not tied to specific implementations.

5 Context within the Master Program

This thesis combines aspects of software engineering, systems-level programming, and security, and therefore fits well within the Software Engineering Master program. It builds in particular on concepts from the following courses:

- **Low-Level Programming**
- **System and Application Security**
- **Digital Forensics**
- **Seminar on Security**
- **Advanced Privacy Enhancing Technologies**

References

[1] Raymond Canzanese, Spiros Mancoridis, and Moshe Kam. System call-based detection of malicious processes. In *2015 IEEE International Conference on Software Quality, Reliability and Security*, pages 119–124, 2015.

[2] Sam Collins, Alex Poulopoulos, Marius Muench, and Tom Chothia. Anti-cheat: Attacks and the effectiveness of client-side defences. In *Proceedings of the 2024 Workshop on Research on Offensive and Defensive Techniques in the Context of Man At The End (MATE) Attacks*, CheckMATE '24, page 30–43, New York, NY, USA, 2024. Association for Computing Machinery.

[3] Giorgia Di Pietro, Daniele Cono D'Elia, and Leonardo Querzoni. Can you run my code? a close look at process injection in windows malware. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, ASIA CCS '25, page 1600–1616, New York, NY, USA, 2025. Association for Computing Machinery.

[4] S. Forrest, S.A. Hofmeyr, A. Somayaji, and T.A. Longstaff. A sense of self for unix processes. In *Proceedings 1996 IEEE Symposium on Security and Privacy*, pages 120–128, 1996.

[5] Mohammed Nasereddin and Raad Al-Qassas. A new approach for detecting process injection attacks using memory analysis. *Int. J. Inf. Secur.*, 23(3):2099–2121, March 2024.